

Efficient composition with DynamicBuffer

Contents

- [Introduction](#)
- [Description](#)
 - [Asynchronous Operations](#)
 - [Problem with Composition](#)
 - [Problem with Exceptions](#)
 - [Problem with Workarounds](#)
- [Solution](#)
- [References](#)

Introduction

This paper describes a design flaw in the specification of the **DynamicBuffer** concept found in [networking.ts], which prevents efficient composition of asynchronous operations accepting an instance of a dynamic buffer. The proposed remedy reverts back some of the differences in the TS from its precursor library Asio (and Boost.Asio) by making **DynamicBuffer** a true storage type.

Description

Most network algorithms accept fixed-size buffers as input, output, or both, in their signatures and contracts. For operations where the caller cannot easily determine ahead of time the storage requirements needed for an algorithm to meet its post-conditions, [networking.ts] introduces the **DynamicBuffer** concept:

A dynamic buffer encapsulates memory storage that may be automatically resized as required, where the memory is divided into two regions: readable bytes followed by writable bytes. [buffer.reqmts.dynamicbuffer]

Readable and writable bytes returned by a dynamic buffer are represented respectively by the **ConstBufferSequence** and **MutableBufferSequence** concepts defined in the TS. These sequences are a special type of range whose element type is a span of bytes. Network algorithms append data to the dynamic buffer in two steps. First, a buffer sequence representing writable bytes is obtained by calling `prepare`. Then, after the algorithm has placed zero or more bytes at the beginning of the sequence the function `commit` is called to move some or all of that data to the sequence representing the readable bytes. The term "move" here is notional: the dynamic buffer concept does not require (nor disallow) a memory move. The following code reads from a socket into a dynamic buffer:

```
string s;  
dynamic_string_buffer b{s}; // dynamic_string_buffer is defined by the TS  
auto const bytes_transferred = socket.read_some(b.prepare(1024)); // read up to 1024 bytes  
b.commit(bytes_transferred);  
assert(s.size() == bytes_transferred);
```

As can be seen in the preceding code, an instance of `dynamic_string_buffer` does not own its storage. Overwhelming feedback from users indicates they wish to use traditional types like `string` or `vector` to store their data. The TS improves on its predecessor Asio by providing the types `dynamic_string_buffer` and `dynamic_vector_buffer` to achieve this goal (subsequent discussions will refer only to `dynamic_string_buffer`, but also apply to `dynamic_vector_buffer`). The TS provides a family of algorithms which allow the code above to be expressed more succinctly by passing the dynamic buffer instead of the buffer sequence representing the writable bytes. To facilitate construction and lifetime management of these dynamic buffers, the TS function `dynamic_buffer` is overloaded for various container types to return a dynamic buffer instance, as seen in this synchronous example:

```
string s;  
auto const bytes_transferred = read(socket, dynamic_buffer(s));  
assert(s.size() == bytes_transferred);
```

Given the semantics of dynamic buffers implied by the wording, instances of dynamic buffers behave more like references to storage types rather than storage types, as copies refer to the same underlying storage. This can be seen in the declaration of `dynamic_string_buffer` which meets the requirements of **DynamicBuffer**:

```
template <typename Elem, typename Traits, typename Allocator>  
class dynamic_string_buffer  
{  
    [...]  
private:  
    std::basic_string<Elem, Traits, Allocator>& string_;  
    std::size_t size_;  
    const std::size_t max_size_;  
};
```

A dynamic string buffer contains a reference to the underlying string. Copies of a dynamic string buffer refer to the same string. Note that the dynamic string buffer also contains some state: the `size_` and `max_size_` data members. This additional metadata informs the dynamic string buffer of the boundaries between the readable and writable bytes, as well as the maximum allowed size of the total of the readable and writable bytes.

Asynchronous Operations

Thus far we have shown synchronous (blocking) examples. Asynchronous algorithms are started by a call to an *initiating function* which returns immediately. When the operation has completed, the implementation invokes a caller-provided function object called a *completion handler* with the result of the operation, usually indicated by an error code and possible additional data such as the number of bytes transacted. Here is a typical signature of an asynchronous TS algorithm which accepts a dynamic buffer instance:

```
// 17.10 [networking.ts::buffer.async.read.until], asynchronous delimited read operations:

template<
    class AsyncReadStream,
    class DynamicBuffer,
    class CompletionToken>
DEDUCED async_read_until(
    AsyncReadStream& s,
    DynamicBuffer&& b,
    char delim,
    CompletionToken&& token);
```

Since the initiating function returns immediately, it is necessary for the asynchronous operation to manage the lifetime of the dynamic buffer parameter. Guidance for doing so is given in the TS:

13.2.7.5 Lifetime of initiating function arguments [async.reqmts.async.lifetime]

1. Unless otherwise specified, the lifetime of arguments to initiating functions shall be treated as follows: [...] the implementation does not assume the validity of the argument after the initiating function completes [...] The implementation may make copies of the argument, and all copies shall be destroyed no later than immediately after invocation of the completion handler.

Given that the requirement for dynamic buffers is that they are **MoveConstructible**, a sensible implementation will make a decay-copy of the argument. An implementation authored by the principal architect of the TS, does precisely that:

```
namespace detail {

template <
    typename AsyncReadStream,
    typename DynamicBuffer,
    typename ReadHandler>
class read_until_delim_op
{
public:
    // Note: DeducedDynamicBuffer will be DynamicBuffer plus cv-ref qualifiers
    template <typename DeducedDynamicBuffer>
    read_until_delim_op(
        AsyncReadStream& stream,
        DeducedDynamicBuffer&& buffers,
        char delim, ReadHandler& handler)
    : [...]
      buffers_(std::forward<DeducedDynamicBuffer>(buffers))
    [...]
    {
    }
    [...]
    void operator()(const std::error_code& ec,
                    std::size_t bytes_transferred, int start = 0);
    [...]
    DynamicBuffer buffers_;
    [...]
};

} // detail

template <
    typename AsyncReadStream,
    typename DynamicBuffer,
    typename ReadHandler>
NET_TS_INITFN_RESULT_TYPE(ReadHandler,
    void (std::error_code, std::size_t))
async_read_until(
    AsyncReadStream& s,
    DynamicBuffer&& buffers,
    char delim,
    ReadHandler&& handler)
{
    // If you get an error on the following line it means that your handler does
    // not meet the documented type requirements for a ReadHandler.
    NET_TS_READ_HANDLER_CHECK(ReadHandler, handler) type_check;

    async_completion<ReadHandler,
        void (std::error_code, std::size_t)> init(handler);

    detail::read_until_delim_op<
        AsyncReadStream,
        typename std::decay<DynamicBuffer>::type,
        NET_TS_HANDLER_TYPE(ReadHandler, void (std::error_code, std::size_t))>(
            s,
            std::forward<DynamicBuffer>(buffers),
            delim,
            init.completion_handler)(std::error_code(), 0, 1);
```

```

    return init.result.get();
}

```

Further evidence that a dynamic buffer represents a lightweight non-owning reference to storage is provided by this quote from LWG issue by the same author as the TS:

Asio's implementation (and the intended specification) performs DECAY_COPY(b) in the async_read, async_write, and async_read_until initiating functions. All operations performed on b are actually performed on that decay-copy, or on a move-constructed descendant of it. **The copy is intended to refer to the same underlying storage and be otherwise interchangeable with the original in every way.** [Ed: emphasis added]

When asynchronous algorithms are constructed from calls to other initiating functions, the result is called a *composed operation*. For example, `async_read` may be implemented in terms of zero or more calls to a stream's `async_read_some` algorithm. Layers of composition may be added to arbitrary degree, permitting algorithms of significant complexity to be developed. A composed operation is typically implemented as a movable function object containing additional state. When the composed operation calls an intermediate initiating function, it passes `std::move(*this)` as the completion handler. This idiom, informally known as "stack-ripping" is used for reasons related to the optimization of memory allocation during asynchronous continuations (and beyond the scope of this paper). Here is the function call operator for the `async_read_until` operation described above:

```

void read_until_op::operator()(const std::error_code& ec,
                               std::size_t bytes_transferred, int start = 0)
{
    ...

    // Start a new asynchronous read operation to obtain more data.
    stream_.async_read_some(buffer_.prepare(bytes_to_read), std::move(*this));

    ...
}

```

Care must be taken when using this idiom, as the memory location of data members belonging to the function object will necessarily change every time an initiating function is invoked. The implication is that lvalue references belonging to any of the data members of the function object may not be passed to intermediate initiating functions. This prevents internal interfaces where dynamic buffers are passed by reference.

Problem with Composition

When only one composed operation handles the dynamic buffer, things seem to work. However, if a composed operation wishes to invoke another composed operation on that dynamic buffer, a problem arises. Consider a composed operation `async_http_read` which accepts a dynamic buffer, and is implemented in terms of zero or more calls to `async_http_read_some`, which also accepts the same dynamic buffer. When `async_http_read_some` is invoked, it takes ownership of the buffer via move. However, if `async_http_read` needs to invoke `async_http_read_some` again to fulfill its contract, it cannot do so without depending on undefined behavior. This is because on the second and subsequent invocations of `async_http_read_some`, the buffer is in a moved-from state, and the specification of **DynamicBuffer** is silent on the condition of moved-from objects. This can be seen in the implementation of the functions described above, which come from the Boost.Beast [2] library:

```

template<class Stream, class DynamicBuffer,
         bool isRequest, class Derived, class Condition,
         class Handler>
class read_http_op : public boost::asio::coroutine
{
    Stream& stream_;
    DynamicBuffer buffer_;
    basic_parser& parser_;
    std::size_t bytes_transferred_ = 0;
    Handler handler_;
    bool cont_ = false;

public:
    [...]

    void
    operator()(
        error_code ec,
        std::size_t bytes_transferred = 0,
        bool cont = true)
    {
        cont_ = cont;
        BOOST_ASIO_CORO_REENTER(*this)
        {
            if(Condition{}(p_))
            {
                BOOST_ASIO_CORO_YIELD
                boost::asio::post(stream_.get_executor(),
                                   bind_handler(std::move(*this), ec));
                goto upcall;
            }
            for(;;)
            {
                // The call to async_http_read_some will produce
                // undefined behavior on the second and subsequent
                // calls, since buffer_ is in the moved-from state:
                //
                BOOST_ASIO_CORO_YIELD
                async_http_read_some(
                    stream_, std::move(buffer_), parser_, std::move(*this));

                if(ec)

```

```

        goto upcall;
        bytes_transferred_ += bytes_transferred;
        if(Condition{}(p_))
            goto upcall;
    }
    upcall:
        handler_(ec, bytes_transferred_);
    }
}
};

```

Problem with Exceptions

Another design problem caused by adding metadata to the dynamic buffer concept is illustrated in the following example code:

```

template<class MutableBufferSequence>
std::size_t read(const MutableBufferSequence&)
{
    throw std::exception{};
}

int main()
{
    std::string s;
    assert(s.empty());
    try
    {
        auto b = boost::asio::dynamic_buffer(s);
        b.commit(read(b.prepare(32)));
    }
    catch(const std::exception&)
    {
        assert(s.empty()); // fails
    }
}

```

While not technically incorrect, it may be surprising to the user that the string contains additional value-initialized data which was not part of the original readable bytes (which in this case was empty). The wording of **DynamicBuffer** and `dynamic_string_buffer` do not address the effect of exceptions on state of the readable bytes in the underlying container at all.

Problem with Workarounds

The problem of composition can be solved using the current specification of **DynamicBuffer** by creating a copyable, reference-counted wrapper which holds a decay-copy of the dynamic buffer argument. Here is a partial sketch of that wrapper:

```

template<class DynamicBuffer>
class shared_buffer
{
    std::shared_ptr<DynamicBuffer> p_;

public:
    template<class DeducedDynamicBuffer>
    explicit shared_buffer(DeducedDynamicBuffer&& b)
        : p_(std::make_shared<DynamicBuffer>(std::forward<DeducedDynamicBuffer>(b)))
    {
    }

    [...]

    mutable_buffers_type prepare(std::size_t n);
    void commit(std::size_t);
};

```

An existing initiating function may be adjusted to use the wrapper instead, as shown:

```

template <
    typename AsyncReadStream,
    typename DynamicBuffer,
    typename ReadHandler>
NET_TS_INITFN_RESULT_TYPE(ReadHandler,
    void (std::error_code, std::size_t))
async_read_until(
    AsyncReadStream& s,
    DynamicBuffer&& buffers,
    char delim,
    ReadHandler&& handler)
{
    // If you get an error on the following line it means that your handler does
    // not meet the documented type requirements for a ReadHandler.
    NET_TS_READ_HANDLER_CHECK(ReadHandler, handler) type_check;

    async_completion<ReadHandler,
        void (std::error_code, std::size_t)> init(handler);

    read_until_delim_op<
        AsyncReadStream,
        shared_buffer<typename std::decay<DynamicBuffer>::type>,

```

```

NET_TS_HANDLER_TYPE(ReadHandler, void (std::error_code, std::size_t))>{
    s,
    shared_buffer<typename std::decay<DynamicBuffer>::type>{std::forward<DynamicBuffer>(buffers)},
    delim,
    init.completion_handler()(std::error_code(), 0, 1);

    return init.result.get();
}

```

This workaround comes at the expense of one or more additional unnecessary dynamic allocations. Utilizing the workaround would sacrifice an important C++ value: zero-cost abstractions.

Solution

The Net.TS predecessors Asio and Boost.Asio offered only the single concrete type `basic_streambuf` which addressed the problem of stream algorithms which need to write a dynamic number of octets. The interface to `basic_streambuf` almost perfectly matches the requirements of **DynamicBuffer**. Field experience showed that users had trouble manipulating buffer sequences, often writing code which performed unnecessary allocate/copy/free cycles to get the data in the format they desired. Users loudly declared the desire to send and receive their data using traditional containers such as `string` and `vector` with which they are already very familiar.

To support the use of strings and vectors in stream algorithms, the **DynamicBuffer** concept is fashioned as a non-owning reference wrapper to a supported container type whose lifetime is managed by the caller. Algorithms which accept dynamic buffer parameters use a forwarding reference, and take ownership via decay-copy. Net.TS provides some functions which accept a reference to a user-managed container and return a suitable wrapper. These functions are designed for ease of use, as they may be invoked directly at call sites:

```

// 16.14 [networking.ts::buffer.dynamic.creation], dynamic buffer creation:

template<class T, class Allocator>
dynamic_vector_buffer<T, Allocator>
dynamic_buffer(vector<T, Allocator>& vec) noexcept;
template<class T, class Allocator>
dynamic_vector_buffer<T, Allocator>
dynamic_buffer(vector<T, Allocator>& vec, size_t n) noexcept;

template<class CharT, class Traits, class Allocator>
dynamic_string_buffer<CharT, Traits, Allocator>
dynamic_buffer(basic_string<CharT, Traits, Allocator>& str) noexcept;
template<class CharT, class Traits, class Allocator>
dynamic_string_buffer<CharT, Traits, Allocator>
dynamic_buffer(basic_string<CharT, Traits, Allocator>& str, size_t n) noexcept;

```

The solution we propose is to change the semantics of **DynamicBuffer** to represent a true storage type rather than a hybrid reference with metadata. Instances of dynamic buffers will be passed by reference, and callers will be required to manage the lifetime of dynamic buffer objects for the duration of any asynchronous operations. As the caller is already responsible for maintaining the lifetime of the stream, and the `string` or `vector` under the old scheme, maintaining the lifetime of the dynamic buffer in the new scheme poses no additional burden. To solve the problem of exceptions, our solution refactors the dynamic buffer wrappers to own their corresponding `string` and `vector`. The caller can only take ownership of the container by calling a member function. The implementation then has the opportunity to adjust the container to reflect the correct state before the caller receives the value.

We believe that there is a tension between ease of use and utility. The current specification provides ease of use but lacks utility, specifically the ability to compose higher level operations out of lower level ones which operate on dynamic buffers. The solution we propose goes in the opposite direction, sacrificing some ease of use in exchange for the ability to compose. The author of this paper favors composition over ease of use for a few reasons. First, because C++ should always prioritize zero-cost abstractions as a primary factor to differentiate itself from other languages. And second, because the entirety of Net.TS provides *low-level abstractions*. Most users should not be concerning themselves with interacting at the raw TCP/IP level. Instead they should be working with high level interfaces such as HTTP client APIs, WebSocket framework APIs, REST SDKs, and the like. These high level interfaces do not exist yet, but with the standardization of low level networking provided in Net.TS we are confident they will arrive in due time.

Note that writing asynchronous code in a clean style which uses all the modern idioms of Asio and Net.TS is quite a difficult endeavor, as evidenced by the distinct lack of open source libraries built on Asio which follow best practices (despite Asio having been available for over a decade). Furthermore the number of people who need to author components which interact directly with low-level interfaces such as Asio or Net.TS should be few in number. Therefore, we favor a Net.TS specification which addresses the needs for writers of specialized middleware and high-level interfaces over ordinary users.

The author of this paper is not entirely satisfied with the proposed solution, and believes that finding the right solution which balances the needs of utility with ease of use is probably best left to the LEWG.

Proposed Wording

This wording is relative to [N4734](#).

[Drafting note: The project editor is kindly asked to replace all occurrences of `DynamicBuffer&&` with `DynamicBuffer&` as indicated by the provided wording changes below. — end drafting note]

1. Modify 16.1 [networking.ts::buffer.synop], header `<experimental/buffer>` synopsis, as indicated:

```

[...]
// 16.11 [networking.ts::buffer.creation], buffer creation:
[...]

template<class T, class Allocator = allocator<T>>

```

```

class dynamic_vector_buffer;

template<class CharT, class Traits = char_traits<CharT>, class Allocator = allocator<CharT>>
class basic_dynamic_string_buffer;

using dynamic_string_buffer = basic_dynamic_string_buffer<char>;

// 16.14 [networking.ts::buffer.dynamic.creation], dynamic buffer creation:

template<class T, class Allocator>
dynamic_vector_buffer<T, Allocator>
dynamic_buffer(vector<T, Allocator>& vec) noexcept;
template<class T, class Allocator>
dynamic_vector_buffer<T, Allocator>
dynamic_buffer(vector<T, Allocator>& vec, size_t n) noexcept;

template<class CharT, class Traits, class Allocator>
dynamic_string_buffer<CharT, Traits, Allocator>
dynamic_buffer(basic_string<CharT, Traits, Allocator>& str) noexcept;
template<class CharT, class Traits, class Allocator>
dynamic_string_buffer<CharT, Traits, Allocator>
dynamic_buffer(basic_string<CharT, Traits, Allocator>& str, size_t n) noexcept;

[...]
```

// 17.5 [networking.ts::buffer.read], synchronous read operations:

```

[...]
```

```

template<class SyncReadStream, class DynamicBuffer>
size_t read(SyncReadStream& stream, DynamicBuffer&& b);
template<class SyncReadStream, class DynamicBuffer>
size_t read(SyncReadStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncReadStream, class DynamicBuffer, class CompletionCondition>
size_t read(SyncReadStream& stream, DynamicBuffer&& b,
            CompletionCondition completion_condition);
template<class SyncReadStream, class DynamicBuffer, class CompletionCondition>
size_t read(SyncReadStream& stream, DynamicBuffer&& b,
            CompletionCondition completion_condition, error_code& ec);

// 17.6 [networking.ts::buffer.async.read], asynchronous read operations:
[...]
```

```

template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
DEDUCED async_read(AsyncReadStream& stream,
                  DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncReadStream, class DynamicBuffer,
        class CompletionCondition, class CompletionToken>
DEDUCED async_read(AsyncReadStream& stream,
                  DynamicBuffer&& b,
                  CompletionCondition completion_condition,
                  CompletionToken&& token);

// 17.7 [networking.ts::buffer.write], synchronous write operations:
[...]
```

```

template<class SyncWriteStream, class DynamicBuffer>
size_t write(SyncWriteStream& stream, DynamicBuffer&& b);
template<class SyncWriteStream, class DynamicBuffer>
size_t write(SyncWriteStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
            CompletionCondition completion_condition);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
            CompletionCondition completion_condition, error_code& ec);

// 17.8 [networking.ts::buffer.async.write], asynchronous write operations:
[...]
```

```

template<class AsyncWriteStream, class DynamicBuffer, class CompletionToken>
DEDUCED async_write(AsyncWriteStream& stream,
                  DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncWriteStream, class DynamicBuffer,
        class CompletionCondition, class CompletionToken>
DEDUCED async_write(AsyncWriteStream& stream,
                  DynamicBuffer&& b,
                  CompletionCondition completion_condition,
                  CompletionToken&& token);

// 17.9 [networking.ts::buffer.read.until], synchronous delimited read operations:

template<class SyncReadStream, class DynamicBuffer>
size_t read_until(SyncReadStream& s, DynamicBuffer&& b, char delim);
template<class SyncReadStream, class DynamicBuffer>
size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
                  char delim, error_code& ec);
template<class SyncReadStream, class DynamicBuffer>
size_t read_until(SyncReadStream& s, DynamicBuffer&& b, string_view delim);
template<class SyncReadStream, class DynamicBuffer>
size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
                  string_view delim, error_code& ec);

```

```
// 17.10 [networking.ts::buffer.async.read.until], asynchronous delimited read operations:
```

```
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
                             DynamicBuffer&& b, char delim,
                             CompletionToken&& token);
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
                             DynamicBuffer&& b, string_view delim,
                             CompletionToken&& token);
```

[...]

2. Modify 16.2.4 [networking.ts::buffer.reqmts.dynamicbuffer], as indicated:

-1- [...]

-2- A type *X* meets the `DynamicBuffer` requirements if it satisfies the requirements of `Destructible` (C++ 2014 [destructible]) and `MoveConstructible` (C++ 2014 [moveconstructible]), as well as the additional requirements listed in Table 14.

3. Modify 16.12 [networking.ts::buffer.dynamic.vector], as indicated:

[...]

```
template<class T, class Allocator = allocator<T>>
class dynamic_vector_buffer
{
public:
    // types:
    using value_type = vector<T, Allocator>;
    using const_buffers_type = const_buffer;
    using mutable_buffers_type = mutable_buffer;

    // constructors:
    dynamic_vector_buffer();
    explicit dynamic_vector_buffer(size_t maximum_size);
    explicit dynamic_vector_buffer(vector<T, Allocator>&& vec) noexcept;
    dynamic_vector_buffer(vector<T, Allocator>&& vec, size_t maximum_size) noexcept;
    dynamic_vector_buffer(dynamic_vector_buffer&& other) = default; noexcept;

    dynamic_vector_buffer& operator=(const dynamic_vector_buffer&) = delete;

    // members:
    size_t size() const noexcept;
    size_t max_size() const noexcept;
    void max_size(size_t maximum_size) noexcept;
    size_t capacity() const noexcept;
    const_buffers_type data() const noexcept;
    mutable_buffers_type prepare(size_t n);
    void commit(size_t n);
    void consume(size_t n);
    span<const T> get() const noexcept;
    value_type release();

private:
    vector<T, Allocator>& value_type vec_; // exposition only
    size_t size_; // exposition only
    const size_t max_size_; // exposition only
};
```

[...]

-2- [...]

-3- [...]

`dynamic_vector_buffer();`

-?- Effects: Default-constructs `vec` . Initializes `size` with 0, and `max_size` with `vec.max_size()`.

`explicit dynamic_vector_buffer(size_t maximum_size);`

-?- Effects: Default-constructs `vec` . Initializes `size` with 0, and `max_size` with `maximum_size`.

`explicit dynamic_vector_buffer(vector<T, Allocator>&& vec) noexcept;`

-4- Effects: Initializes `vec_` with `std::move(vec)`, `size_` with `vec.size()`, and `max_size_` with `vec.max_size()`.

`dynamic_vector_buffer(vector<T, Allocator>&& vec, size_t maximum_size) noexcept;`

-5- Requires: `vec.size() <= maximum_size`.

-6- Effects: Initializes `vec_` with `std::move(vec)`, `size_` with `vec.size()`, and `max_size_` with `maximum_size`.

`dynamic_vector_buffer(dynamic_vector_buffer&& other) noexcept;`

-?- Effects: Initializes `vec` with `std::move(other.vec_)`, `size` with `other.size`, and `max_size` with `other.max_size`. Then calls `other.vec_.clear()`; and assigns 0 to `other.size`.

[...]

`size_t max_size() const noexcept;`

-8- Returns: `max_size_`.

`void max_size(size_t maximum_size);`

-?- Requires: `size() <= maximum_size`.

-?- Effects: Performs `max_size = maximum_size`.

[...]

`void consume(size_t n);`

-15- Effects: [...]

`span<const T> get() const noexcept;`

-?- Returns: `span<const T>(vec_.data(), size_)`.

`value_type release();`

-?- Effects: Equivalent to:

```
vec_.resize(size_);
value_type tmp = std::move(vec_);
vec_.clear();
size_ = 0;
return tmp;
```

4. Modify 16.13 [networking.ts::buffer.dynamic.string], as indicated:

```
template<class CharT, class Traits = char_traits<CharT>, class Allocator = allocator<CharT>>
class basic_dynamic_string_buffer
{
public:
    // types:
    using value_type = basic_string<CharT, Traits, Allocator>;
    using const_buffers_type = const_buffer;
    using mutable_buffers_type = mutable_buffer;

    // constructors:
    basic_dynamic_string_buffer();
    explicit basic_dynamic_string_buffer(size_t maximum_size);
    explicit basic_dynamic_string_buffer(basic_string<CharT, Traits, Allocator> value_type&& str) noexcept;
    basic_dynamic_string_buffer(basic_string<CharT, Traits, Allocator> value_type&& str, size_t maximum_size) noexcept;
    basic_dynamic_string_buffer(basic_dynamic_string_buffer&& other) = default;
    basic_dynamic_string_buffer& operator=(basic_dynamic_string_buffer&&) = delete;

    // members:
    size_t size() const noexcept;
    size_t max_size() const noexcept;
    void max_size(size_t maximum_size) noexcept;
    size_t capacity() const noexcept;
    const_buffers_type data() const noexcept;
    mutable_buffers_type prepare(size_t n);
    void commit(size_t n) noexcept;
    void consume(size_t n);
    basic_string_view<CharT, Traits> get() const noexcept;
    value_type release();

private:
    basic_string<CharT, Traits, Allocator>& value_type str_; // exposition only
    size_t size_; // exposition only
    const size_t max_size_; // exposition only
};
```

-2- [...]

-3- [...]

`basic_dynamic_string_buffer();`

-?- Effects: Default-constructs `str`. Initializes `size` with 0, and `max_size` with `str.max_size()`.

`explicit basic_dynamic_string_buffer(size_t maximum_size);`

-?- Effects: Default-constructs `str`. Initializes `size` with 0, and `max_size` with `maximum_size`.

[...]

-14- The program shall ensure **both** the AsyncReadStream object stream **and the DynamicBuffer object b are is** valid until the completion handler for the asynchronous operation is invoked.

8. Modify 17.7 [networking.ts::buffer.write], as indicated:

[...]

```
template<class SyncWriteStream, class DynamicBuffer>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b);
template<class SyncWriteStream, class DynamicBuffer>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b, error_code& ec);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition);
template<class SyncWriteStream, class DynamicBuffer, class CompletionCondition>
    size_t write(SyncWriteStream& stream, DynamicBuffer&& b,
        CompletionCondition completion_condition,
        error_code& ec);
```

[...]

9. Modify 17.8 [networking.ts::buffer.async.write], as indicated:

[...]

```
template<class AsyncWriteStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_write(AsyncWriteStream& stream,
        DynamicBuffer&& b, CompletionToken&& token);
template<class AsyncWriteStream, class DynamicBuffer, class CompletionCondition,
    class CompletionToken>
    DEDUCED async_write(AsyncWriteStream& stream,
        DynamicBuffer&& b,
        CompletionCondition completion_condition,
        CompletionToken&& token);
```

[...]

-14- The program shall ensure both the AsyncWriteStream object stream and the **DynamicBuffer object b** ~~memory associated with the dynamic-buffer-b~~ are valid until the completion handler for the asynchronous operation is invoked.

10. Modify 17.9 [networking.ts::buffer.read.until], as indicated:

```
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, char delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
        char delim, error_code& ec);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b, string_view delim);
template<class SyncReadStream, class DynamicBuffer>
    size_t read_until(SyncReadStream& s, DynamicBuffer&& b,
        string_view delim, error_code& ec);
```

[...]

11. Modify 17.10 [networking.ts::buffer.async.read.until], as indicated:

```
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
        DynamicBuffer&& b, char delim,
        CompletionToken&& token);
template<class AsyncReadStream, class DynamicBuffer, class CompletionToken>
    DEDUCED async_read_until(AsyncReadStream& s,
        DynamicBuffer&& b, string_view delim,
        CompletionToken&& token);
```

[...]

-6- The program shall ensure **both** the AsyncReadStream object stream **and the DynamicBuffer object b are is** valid until the completion handler for the asynchronous operation is invoked.

References

[1] <http://cplusplus.github.io/LWG/lwg-active.html#3072>

[2] <https://github.com/boostorg/beast>